



**UNIVERSIDAD
DON BOSCO**



D S P 4 0 4

Desarrollo de Aplicaciones con Software Propietario

Universidad de El Salvador

S e m a n a 1



Introducción a C# y al entorno .NET

.NET 10 LTS

C# 14

Visual Studio 2026

Temas de hoy:

- Características del entorno de C# / .NET
- Conceptos básicos del lenguaje
- Variables, constantes y tipos de datos
- Expresiones y operadores
- Entrada y salida por consola

¿Qué lograrás al terminar esta clase?



Reconoces el entorno .NET y para qué sirve C#

Sabrás dónde encaja .NET en el mundo del desarrollo.

Declaras variables y constantes con el tipo correcto

int, double, string, bool, char — y cuándo usar var.

Escribes expresiones aritméticas y de texto

Operas valores, los combinas y predices el resultado.

Usas Console.WriteLine y Console.ReadLine

Puedes mostrar datos en pantalla y pedirlos al usuario.

Aplicas interpolación de strings (\$"...")

Construyes mensajes dinámicos de forma limpia.

¿Qué es .NET?



El ecosistema donde vive C#



.NET es una plataforma de desarrollo

- Creada por Microsoft, ahora es código abierto
- Gratis: descarga, usa y distribuye sin costo
- Multiplataforma: corre en Windows, Linux y macOS
- Incluye herramientas para compilar, ejecutar y depurar
- .NET es el 'motor'; C# es el lenguaje que escribimos

💡 Analogía: .NET es como el motor de un auto. C# es el volante con el que tú lo manejas.

.NET 10

LTS

Soporte hasta Nov 2028
← Usaremos esta versión

.NET 9

STS

Soporte 18 meses

.NET Framework

Legacy

Versión antigua (Windows only)
No usar en proyectos nuevos

¿Qué es C# y para qué sirve?



Tipado fuerte y estático

Cada variable tiene un tipo fijo. El compilador detecta errores antes de ejecutar.



Orientado a Objetos (POO)

Todo se organiza en clases y objetos. (Esto lo veremos a partir de la semana 5.)



Compilado a bytecode

C# se convierte a IL (Intermediate Language) y luego la CPU lo ejecuta vía .NET Runtime.



Para web, escritorio y más

Con ASP.NET Core (web), WinForms/WPF (escritorio), .NET MAUI (móvil/iOS/Android).



Sintaxis familiar

Parecido a Java y muy cercano a JavaScript en su estructura de control (if, for, while).



Completamente gratuito

.NET SDK, Visual Studio Community y todas las herramientas que usaremos son gratis.

El entorno: Visual Studio 2026 Community

Lo que verán cuando abran VS por primera vez

Crear un proyecto de consola

1

Abrir Visual Studio 2026

Archivo → Nuevo → Proyecto

2

Buscar 'Console App'

Plantilla de C# (no Visual Basic)

3

Nombre del proyecto

Sin espacios ni caracteres especiales

4

Framework: .NET 10 (LTS)

Siempre elegir esta versión

5

¡Crear!

VS genera la estructura automáticamente

Estructura del proyecto

```

MiPrimeraApp/
├── Program.cs
│   └── ← ¡Tu código va aquí!
├── .csproj
│   └── ← Configuración del proyecto
├── bin/
│   └── ← Compilado (auto)
└── obj/
    └── ← Temporales (auto)
```

💡 En .NET 10 el archivo Program.cs es minimalista:
NO necesitas escribir 'class Program' ni 'static void Main()'
Eso lo hace el compilador automáticamente (Top-Level Statements).

El primer programa — Program.cs



CS

```
// Program.cs — .NET 10 (Top-Level Statements)

Console.WriteLine("¡Hola, DSP404!");
Console.WriteLine("Bienvenido a C#.");
```

Salida en consola:

```
¡Hola, DSP404!
Bienvenido a C#.
```

¿Cómo ejecutar el programa?



F5

Ejecutar con depurador (puedes ver errores paso a paso)



Ctrl+F5

Ejecutar sin depurador (más rápido, para pruebas)



Ctrl+B

Solo compilar sin ejecutar (verificar errores)



Shift+F5

Detener la ejecución



UNIVERSIDAD
DON BOSCO

Bloque 2

Variables, Constantes y Tipos de Datos

El bloque fundamental de cualquier programa

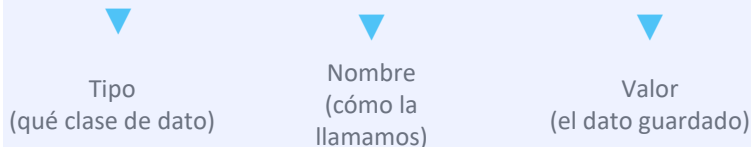
¿Qué es una variable?

Una caja con nombre que guarda un valor



```
int edad = 20;
```

tipo nombre valor



Reglas de las variables



Se declaran antes de usarse

El tipo va primero, luego el nombre.



El valor puede cambiar

De ahí el nombre 'variable'.



El tipo **NO** puede cambiar

Una int siempre guarda enteros.



Nombres en camelCase

edadAlumno, precioTotal, nombreCliente.



No empezar con número

1variable → ❌ variable1 → ✅

Tipos de datos primitivos en C#



UNIVERSIDAD
DON BOSCO

Los bloques básicos de información

Tipo	¿Qué guarda?	Ejemplo de declaración	Rango / Nota
int	Número entero	<code>int edad = 20;</code>	-2,147M a 2,147M
long	Entero grande	<code>long poblacion = 80000000000L;</code>	$\pm 9.2 \times 10^{18}$
double	Decimal (precisión)	<code>double precio = 19.99;</code>	~15-17 dígitos sig.
float	Decimal (liviano)	<code>float temp = 36.5f;</code>	~6-9 dígitos sig.
decimal	Decimal financiero	<code>decimal saldo = 1500.50m;</code>	28-29 dígitos — ¡Dinero!
string	Texto (cadena)	<code>string nombre = "Ana";</code>	Comillas dobles ""
char	Un carácter	<code>char letra = 'A';</code>	Comillas simples "
bool	Verdadero/Falso	<code>bool activo = true;</code>	Solo true o false



Para empezar, los que más usarás: **int**, **double**, **string** y **bool**.

Declarar y asignar variables — sintaxis completa



UNIVERSIDAD
DON BOSCO

CS

```
// FORMA 1: Declarar y asignar en la misma línea (lo más común)
int edad = 20;
string nombre = "Carlos";
double promedio = 8.75;
bool aprobado = true;
```

```
// FORMA 2: Declarar primero, asignar después
int nota;
nota = 95;           // <-- asignación posterior
```

```
// FORMA 3: Declarar varias del mismo tipo en una línea
int x = 0, y = 0, z = 0;
```

```
// FORMA 4: Usar 'var' — el compilador deduce el tipo
var precio = 19.99;           // es double
var ciudad = "San Salvador"; // es string
var cantidad = 5;             // es int
```

```
// ¿Puedo cambiar el tipo si uso var?
var item = "Libro";
item = 42; // ❌ ERROR: el tipo ya fue fijado como string
```

Reglas del operador =



= asigna (no es 'igual a')

Lee: 'la variable recibe el valor'



El valor va a la derecha

edad = 20; (no 20 = edad)



Puede reasignarse

edad = 20; → edad = 21; (válido)



var fija el tipo al declarar

var x = 5; x = 3.14 → ERROR

Constantes — valores que no deben cambiar

Una constante es un valor fijo que no debe cambiar

CS

```
// Una constante se declara con la palabra clave 'const'
const double PI = 3.14159265;
const int DIAS_SEMANA = 7;
const string NOMBRE_APP = "Sistema de Inventario";

// Intentar cambiarla → ERROR de compilación
PI = 3.0; // ❌ Error: Cannot assign to 'PI' because it is read-only

// — USO EN CÁLCULO —
double radio = 5.0;
double area = PI * radio * radio;
Console.WriteLine($"Área del círculo: {area:F2}");

// — CONVENCIÓN: nombres en UPPER_SNAKE_CASE —
const int MAX_ALUMNOS = 40;
const double IVA = 0.13; // El IVA del 13% no cambia (en El Salvador)
```



Usar const cuando:

El valor ES fijo: PI, IVA, número de días, límites



No usar const cuando:

El valor puede cambiar: precio, edad, nombre de usuario



UNIVERSIDAD
DON BOSCO

Bloque 3

Expresiones y Operadores

Hacer que los datos trabajen juntos

Operadores aritméticos

Las operaciones matemáticas del lenguaje

Op	Nombre	Ejemplo	Resultado
+	Suma	<code>int r = 8 + 5;</code>	<code>r = 13</code>
-	Resta	<code>int r = 10 - 3;</code>	<code>r = 7</code>
*	Multiplicación	<code>int r = 4 * 6;</code>	<code>r = 24</code>
/	División	<code>int r = 10 / 3;</code>	<code>r = 3 (¡entera!)</code>
/	División decimal	<code>double r = 10.0 / 3;</code>	<code>r = 3.3333...</code>
%	Módulo (residuo)	<code>int r = 10 % 3;</code>	<code>r = 1</code>

División entera (trampa clásica):

Si ambos operandos son int, C# hace división ENTERA y descarta el decimal.

CS

```
int a=10, b=3;  
Console.WriteLine(a/b);      // 3   (no 3.33)  
Console.WriteLine((double)a/b); // 3.3333
```

Operadores de incremento y asignación compuesta

CS

```
int contador = 0;

// Incremento y decremento
contador++;           // equivale a: contador = contador + 1
→ 1
contador++;           // → 2
contador--;           // → 1

// Asignación compuesta (shorthand)
int precio = 100;
precio += 50;          // precio = precio + 50    → 150
precio -= 20;          // precio = precio - 20    → 130
precio *= 2;           // precio = precio * 2     → 260
precio /= 4;           // precio = precio / 4     → 65
precio %= 10;          // precio = precio % 10    → 5

// ¿Dónde se usa esto?
int alumnos = 0;
alumnos++;             // cuando llega un alumno al salón
alumnos++;
alumnos++;
Console.WriteLine($"Alumnos presentes: {alumnos}"); // 3
```

¿Por qué existen?

- Menos código que escribir
- Más legible para otros programadores
- Muy usados en ciclos (semana 2)
- `i++` es la forma idiomática de avanzar un contador



Truco para recordar:
`x += 5` → 'x recibe 5 más'



UNIVERSIDAD
DON BOSCO

Bloque 4

Entrada, Salida e Interpolación

Cómo el programa habla con el usuario

Salida por consola — Console.Write y Console.WriteLine

CS

```
// WriteLine → imprime y salta a la siguiente línea
Console.WriteLine("Primera línea");
Console.WriteLine("Segunda línea");
// Salida:
// Primera línea
// Segunda línea

// Write → imprime SIN saltar de línea
Console.Write("Hola ");
Console.Write("Mundo");
Console.WriteLine("!");
// Salida:
// Hola Mundo!

// Imprimir una línea vacía (solo el salto)
Console.WriteLine();

// Imprimir valores de variables
int edad = 20;
Console.WriteLine(edad);           // 20
Console.WriteLine("Edad: " + edad); // Edad: 20 (concatenación)
```

Comparación

Método	Salta línea
Console.Write	✗ No
Console.WriteLine	✓ Sí



Analogía:

Write = dictar sin pausa

WriteLine = dictar y hacer pausa

Secuencias de escape útiles:

`\n`

Nueva línea

`\t`

Tabulación

`\"`

Comilla doble dentro del string

`\\`

Backslash literal

Entrada por consola — Console.ReadLine()

ReadLine SIEMPRE devuelve string — hay que convertir

```
CS
// ReadLine() lee lo que el usuario escribe y devuelve
// un string
Console.Write("Escribe tu nombre: ");
string nombre = Console.ReadLine();    // "Ana" → ya
// es string 

// — PROBLEMA: ¿cómo leer un número?
// -----
Console.Write("Escribe tu edad: ");
string entrada = Console.ReadLine();    // devuelve
"20" (texto)

// Opción A: Parse (lanza excepción si el texto no es
// número)
int edad = int.Parse(entrada);

// Opción B: Convert (equivalente, más explícito)
int edad2 = Convert.ToInt32(entrada);

// Opción C: TryParse (segura, no lanza excepción)
bool exito = int.TryParse(entrada, out int edad3);
```

¿Por qué siempre string?

ReadLine lee lo que el usuario escribe en el teclado.
El teclado solo produce texto.
Es el programa quien decide qué tipo necesita.

Error clásico:

```
CS
int x =
Console.ReadLine();
//  ERROR: no se puede
// convertir
```

```
CS
int x = int.Parse(
    Console.ReadLine());
// 
```

Interpolación de strings — la forma moderna de construir texto



Tres formas de construir un mensaje:

Forma 1 — Concatenación con + (antigua, propensa a errores)

```
string nombre = "María";  
int edad = 22;  
Console.WriteLine("Hola, " + nombre + ". Tienes " + edad + "  
años.");
```

Simple de entender al inicio

Difícil de leer, fácil olvidar espacios, concatenar números puede confundir

Forma 2 — String.Format (legado, .NET 1.x)

```
Console.WriteLine("Hola, {0}. Tienes {1} años.", nombre,  
edad);
```

Permite reutilizar el formato

Los índices {0},{1} son difíciles de rastrear en cadenas largas

Forma 3 ★ — Interpolación con \$"..." (moderna — C# 6+)

```
Console.WriteLine($"Hola, {nombre}. Tienes {edad} años.");
```

✅ Legible, expresiones directas, se puede formatear

Requiere \$ antes de las comillas (fácil de olvidar al inicio)

Interpolación — expresiones, formato y casos especiales

CS

```
string nombre = "Luis";
int edad      = 19;
double nota   = 8.755;
double precio = 1599.9;

// — Expresión matemática DENTRO del string —
Console.WriteLine($"El doble de tu edad es {edad * 2}");
// → El doble de tu edad es 38

// — Formato numérico —
Console.WriteLine($"Nota: {nota:F2}"); // F2 = 2 decimales → 8.76
Console.WriteLine($"Nota: {nota:F0}"); // F0 = entero      → 9
Console.WriteLine($"Precio: {precio:C}"); // C = moneda local → $1,599.90

// — Llamada a método dentro del string —
Console.WriteLine($"Nombre en mayúscula: {nombre.ToUpper()}");
// → LUIS

// — Expresión ternaria (anticipo) —
Console.WriteLine($"Estado: {(nota >= 6 ? "Aprobado" : "Reprobado")}");
// → Estado: Aprobado

// — Llaves literales (escapar con doble llave) —
Console.WriteLine($"Sintaxis: {{variable}}");
// → Sintaxis: {variable}
```

Interpolación — construyendo paso a paso

Ejemplo guiado: de datos sueltos a mensaje completo

Datos de partida:

cs

```
string producto = "Laptop";  
int cantidad    = 3;  
double precio   = 599.99;
```

↓ *¿Cómo mostramos esto?*

✗ Sin interpolación

```
Console.WriteLine("Producto: " + producto  
+ " | Cant: " + cantidad  
+ " | Total: $" + (cantidad*precio));
```

✓ Con interpolación

```
Console.WriteLine(  
    $"Producto: {producto} | Cant: {cantidad} |  
    Total: ${cantidad*precio}");
```

★ Con formato

```
double total = cantidad * precio;  
Console.WriteLine(  
    $"Producto: {producto}\n  Cantidad: {cantidad}\n  Total: {total:C}");
```



Tarjeta de Presentación

Ejercicio explicado — 'Tarjeta de Presentación'

CS

```
// — TARJETA DE PRESENTACIÓN —  
// Archivo: Program.cs | Proyecto Console App | .NET 10  
  
// PASO 1: Encabezado  
Console.WriteLine("=====");  
Console.WriteLine("  BIENVENIDO A C# — DSP404  ");  
Console.WriteLine("=====");  
Console.WriteLine();  
  
// PASO 2: Leer nombre  
Console.Write("¿Tu nombre completo? ");  
string nombre = Console.ReadLine();  
  
// PASO 3: Leer edad (ReadLine → string → int.Parse)  
Console.Write("¿Tu edad? ");  
int edad = int.Parse(Console.ReadLine());  
  
// PASO 4: Leer carrera  
Console.Write("¿Qué estudias? ");  
string carrera = Console.ReadLine();  
  
// PASO 5: Calcular  
const int ANIO_ACTUAL = 2026;  
int anioNac = ANIO_ACTUAL - edad;  
  
// PASO 6: Mostrar con interpolación y formato  
Console.WriteLine();  
Console.WriteLine("=====");  
Console.WriteLine($"  Hola, {nombre}!");  
Console.WriteLine($"  Tienes {edad} años.");  
Console.WriteLine($"  Estudias {carrera}.");  
Console.WriteLine($"  Naciste aprox. en {anioNac}.");  
Console.WriteLine("=====");
```



UNIVERSIDAD
DON BOSCO



¡Tu turno! — Calculadora Básica

30 minutos | Individual o en parejas
Lee el enunciado en el siguiente slide



EJERCICIO — Calculadora de Operaciones Básicas



Crea un programa de consola que haga lo siguiente:

Muestra el título: "=== CALCULADORA BÁSICA ==="

Pide al usuario: su nombre, número A (entero) y número B (entero)

Calcula: suma, resta, multiplicación, división entera y residuo (módulo)

Muestra los 5 resultados con el nombre del usuario, usando interpolación

Despide al usuario con un mensaje que incluya su nombre

Ejemplo de salida:

```
=== CALCULADORA BÁSICA ===  
--- Resultados para María ---  
Suma: 10 + 3 = 13  
División: 10 / 3 = 3  
¡Gracias, María!
```

Pistas:

- Usa `int.Parse(Console.ReadLine())` para los números
- Interpolación: `$"Suma: {a} + {b} = {a+b}"`
- El operador `%` te da el residuo de la división
- Extensión: ¿qué pasa si `B = 0`? (división por cero)

Conversión de tipos (Type Casting)

Cómo transformar un tipo en otro

CS

```
// — CONVERSIÓN IMPLÍCITA (automática, sin pérdida) —————
int entero    = 42;
double decimal2 = entero;    // ✓ int → double (sin perder info)
Console.WriteLine(decimal2); // 42.0

// — CONVERSIÓN EXPLÍCITA (cast) — puede perder datos —————
double pi     = 3.14159;
int piEntero = (int)pi;      // ✓ pero pierde el decimal
Console.WriteLine(piEntero); // 3

// — PARSE — de string a número —————
string texto = "150";
int numero   = int.Parse(texto);    // ✓ "150" → 150
double valor = double.Parse("3.99"); // ✓ "3.99" → 3.99

// — CONVERT —————
int a    = Convert.ToInt32("25");
double b = Convert.ToDouble("8.5");
string c = Convert.ToString(99);    // 99 → "99"

// — TOSTRING — cualquier tipo a string —————
int n = 500;
string s2 = n.ToString();           // "500"
string s3 = n.ToString("C");        // "$500.00" (formato moneda)
```

Jerarquía de tipos numéricos

decimal

Más preciso

double

↑ implícito

float

↑ implícito

long

↑ implícito

int

Más usado

short

↓ requiere cast

Errores comunes en la primera semana

Aprende a leer los mensajes de error

❌ CS0029 — Tipos incompatibles

```
❌ int edad = Console.ReadLine();
```

🔴 Error: Cannot implicitly convert type 'string' to 'int'

```
✅ int edad =  
int.Parse(Console.ReadLine());
```

❌ CS1002 — Punto y coma faltante

```
❌ Console.WriteLine("Hola")
```

🔴 Error: ; expected

```
✅ Console.WriteLine("Hola");
```

❌ FormatException — Parse con texto no numérico

```
❌ int.Parse("abc"); // el usuario
```

🔴 Error: Input string was not in a correct format

```
✅ int.TryParse(entrada, out int n);
```



UNIVERSIDAD
DON BOSCO



Quiz de Diagnóstico

10 preguntas — ¿Cuánto sabías ya de programación?
Responde individualmente en tu dispositivo

Quiz de Diagnóstico — Semana 1

Escanea el código QR o ingresa el enlace en tu navegador

*[Aquí va el código QR]
Reemplaza esta imagen
con el QR de Quizizz*

[Pega aquí el enlace de la sesión: quizizz.com/join?gc=XXXXXX]

Resumen de la Semana 1

- ✓ **.NET 10 y C#:** Plataforma gratuita y multiplataforma. C# es tipado fuerte y estático.
- ✓ **Variables y tipos:** int, double, string, bool, char. El tipo no cambia; el valor sí.
- ✓ **Constantes:** const para valores que nunca deben cambiar (PI, IVA, MAX_ALUMNOS).
- ✓ **Expresiones:** + - * / % ++ -- += Ojo con la división entera int/int.
- ✓ **Console.WriteLine/Write:** Write no salta línea; WriteLine sí. ReadLine devuelve siempre string.
- ✓ **int.Parse():** Convierte el string leído de consola al tipo numérico que necesitamos.
- ✓ **Interpolación \$"...":** Construye mensajes con variables: \$"Hola, {nombre}". Formato con :F2, :C.

Para la próxima clase — Tarea y práctica



Tarea:



- 1 Instalar .NET 10 SDK y Visual Studio 2026 Community (si aún no lo tienen)
- 2 Completar el ejercicio de la calculadora si no terminaron en clase
- 3 Explorar la Unidad 1 de la plataforma de práctica (enlace en el aula virtual)



Plataforma de práctica — Unidad 1 disponible:

- | | |
|--|---|
| ☒ Opción múltiple | ☒ Completar el código |
| ☒ Predecir la salida | ☒ Emparejar términos |

[Enlace a la plataforma: pega aquí el URL]



Thank You
for your attention





UNIVERSIDAD DON BOSCO

Ejercicio explicado — 'Tarieta de Presentación'

udb.edu.sv



UDBelsalvador

